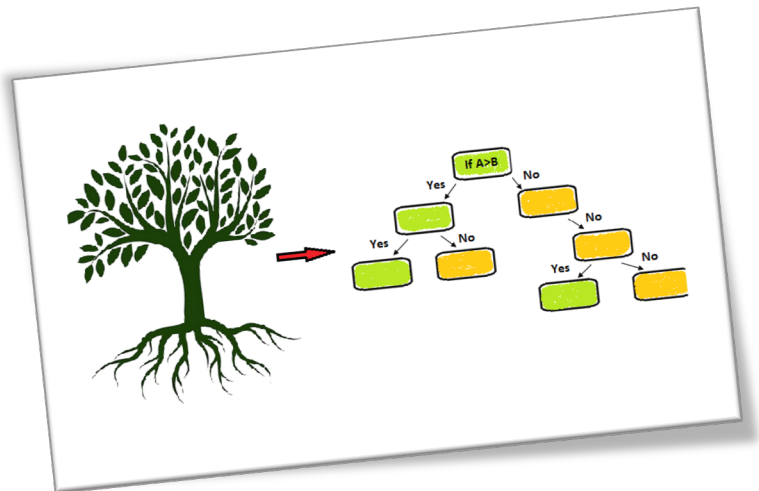
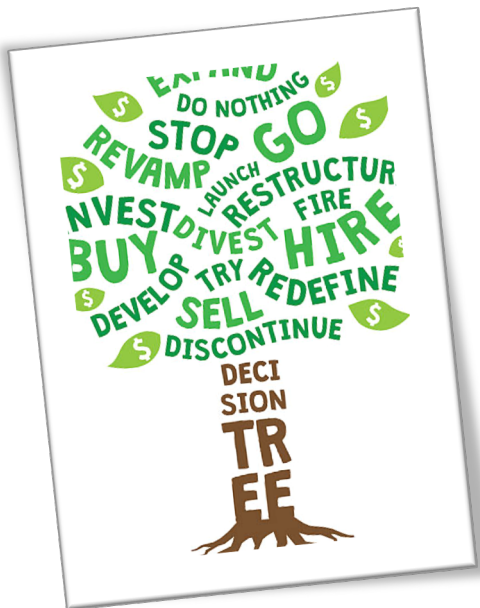


Course Machine Learning

Course #5 – Decision Tree

Professor
Abdelouahab Moussaoui

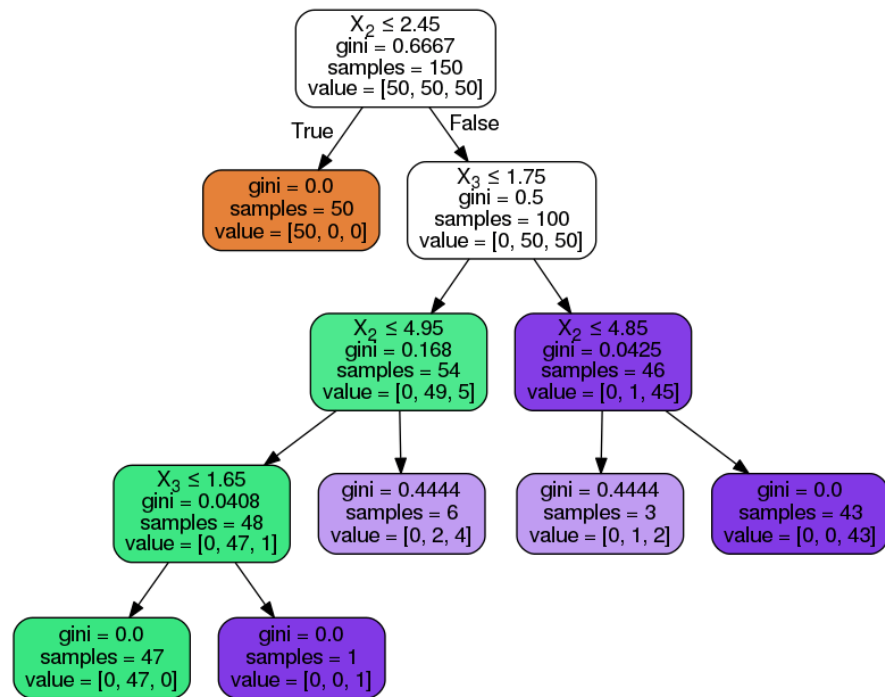


1 — Definitions and concepts

1 — Introduction

The *decision tree algorithm* is a popular supervised *machine learning* algorithm for its simple approach to dealing with complex datasets. Decision trees get the name from their resemblance to a tree that includes roots, branches and leaves in the form of nodes and edges. They are used for decision analysis much like a flowchart of if-else based decisions that lead to the required prediction. The tree learns these if-else decision rules to split the *data set* to make a tree-like model.

Decision trees find their usage in the prediction of discrete results for classification problems and continuous numeric results for regression problems. There are many different algorithms developed over the years, like *CART*, *C4.5* and ensembles, such as *random forest* and *Gradient Boosted Trees*.

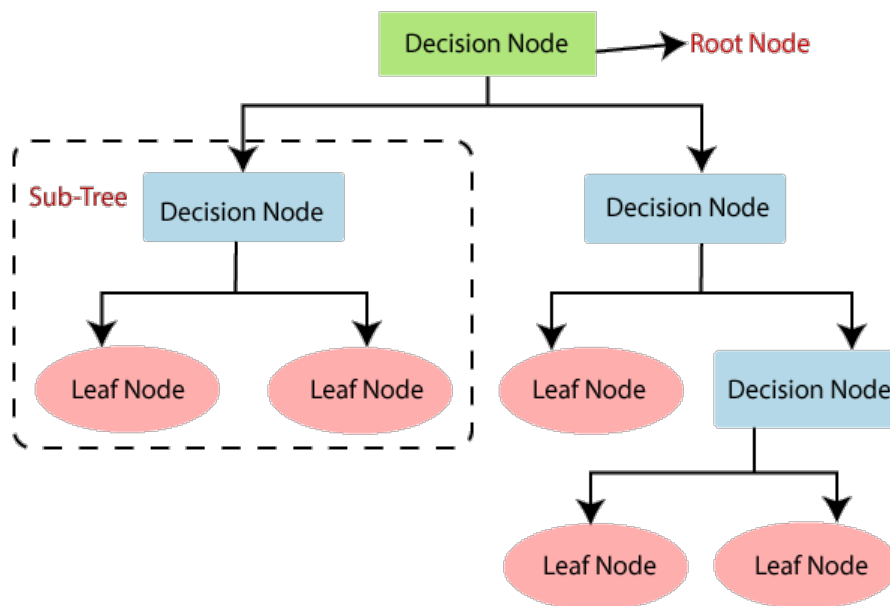


Decision tree example

2 — Characteristics

- Decision Tree is a **Supervised learning technique** that can be used for both *classification* and *Regression* problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome**.
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
- It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.**
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- Below diagram explains the general structure of a decision tree:

Note: A decision tree can contain categorical data (YES/NO) as well as numeric data.



3 — Why use Decision Trees?

There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

- *Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.*
- *The logic behind the decision tree can be easily understood because it shows a tree-like structure.*

4 — Decision Tree Terminologies

- **Root Node:** Root node is from where the decision tree starts. It is the starting node at the top of the decision tree that contains all the attribute values. It represents the entire dataset, which further gets divided into two or more homogeneous sets. Root node splits into decision nodes based on the decision rules that the algorithm has learnt.
- **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node. Leaf nodes are terminal nodes that represent the target prediction. These nodes do not split any further.
- **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions. In binary splits, the branches denote true and false paths.
- **Branch/Sub Tree:** A tree formed by splitting the tree. Branches are connectors between nodes that correspond to the values of attributes. Internal nodes are decision nodes between root node and leaf nodes that correspond to decision rules and their answer paths. Nodes denote questions and branches show paths based on relevant answers to those questions.
- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.

- **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

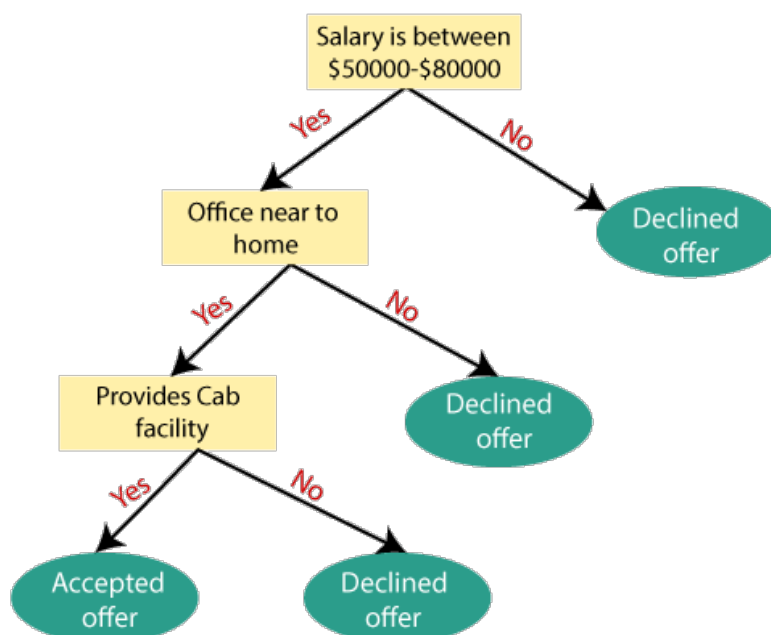
5 — How does the Decision Tree algorithm Work?

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- **Step-1:** Begin the tree with the root node, says S , which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:



2 — Attribute Selection Measures

While implementing a *Decision tree*, the main issue arises that how to *select the best attribute* for the *root node* and for *sub-nodes*. So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM**. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:

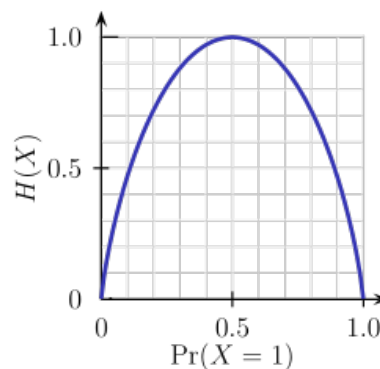
- *Entropy,*
- *Information gain,*
- *Gini index,*
- *Gain Ratio,*
- *Reduction in Variance,*
- *Chi-Square*

These criteria will calculate values for every attribute. The values are sorted, and attributes are placed in the tree by following the order i.e, the attribute with a high value(in case of information gain) is placed at the root.

While using Information Gain as a criterion, we assume attributes to be categorical, and for the Gini index, attributes are assumed to be continuous.

1 — Entropy

Entropy is a measure of the randomness in the information being processed. The higher the entropy, the harder it is to draw any conclusions from that information. Flipping a coin is an example of an action that provides information that is random.



From the above graph, it is quite evident that the entropy $H(X)$ is zero when the probability is either 0 or 1. The Entropy is maximum when the probability is 0.5 because it projects perfect randomness in the data and there is no chance if perfectly determining the outcome.

ID3 follows the rule — A branch with an entropy of zero is a leaf node and A brach with entropy more than zero needs further splitting.

Mathematically Entropy for 1 attribute is represented as:

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5

$$\begin{aligned} \text{Entropy(PlayGolf)} &= \text{Entropy}(5,9) \\ &= \text{Entropy}(0.36, 0.64) \\ &= -(0.36 \log_2 0.36) - (0.64 \log_2 0.64) \\ &= 0.94 \end{aligned}$$

Where $S \rightarrow$ Current state, and $P_i \rightarrow$ Probability of an event i of state S or Percentage of class i in a node of state S .

Mathematically Entropy for multiple attributes is represented as:

$$E(T, X) = \sum_{c \in X} P(c)E(c)$$

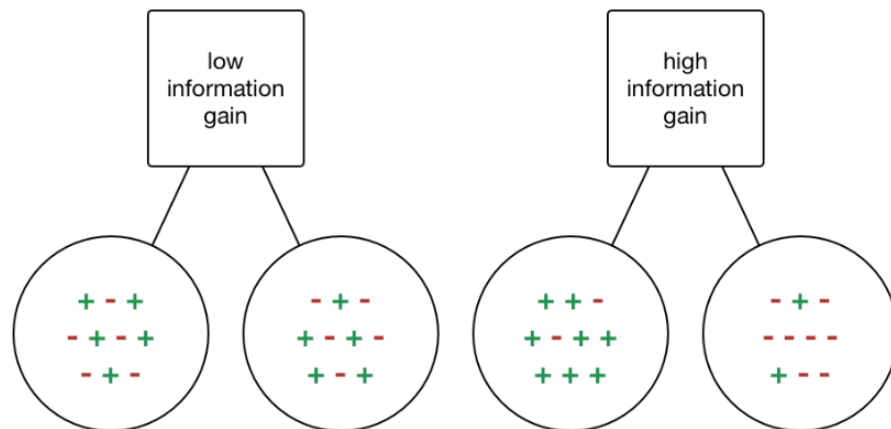
		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14

$$\begin{aligned} E(\text{PlayGolf}, \text{Outlook}) &= P(\text{Sunny}) * E(3,2) + P(\text{Overcast}) * E(4,0) + P(\text{Rainy}) * E(2,3) \\ &= (5/14) * 0.971 + (4/14) * 0.0 + (5/14) * 0.971 \\ &= 0.693 \end{aligned}$$

where $T \rightarrow$ Current state and $X \rightarrow$ Selected attribute

2 — Information Gain

Information gain or **IG** is a statistical property that measures how well a given attribute separates the training examples according to their target classification. Constructing a decision tree is all about finding an attribute that returns the highest information gain and the smallest entropy.



Information Gain Example

Decision Trees in Machine Learning

Trees occupy an important place in the life of man. The trees provide us flowers, fruits, fodder for animals, wood for fire and furniture and provide cool shadow from scorching sun. They give us so many such good things and yet expect nothing in return. Besides being such an important element for the survival of human beings, trees have also inspired wide variety of algorithms in **Machine Learning** both **classification** and **regression**.

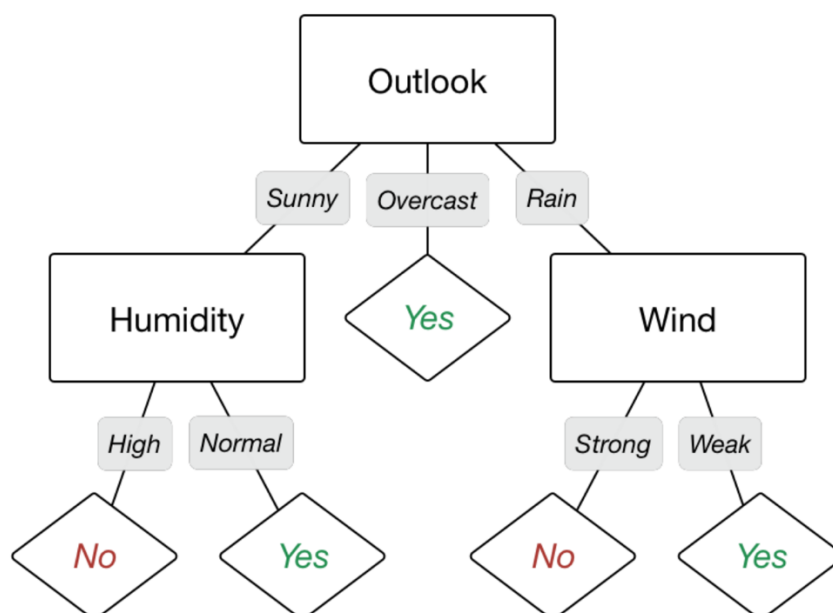
Representation of Algorithm as a Tree

Decision Tree learning algorithm generates decision trees from the training data to solve classification and regression problem. Consider you would like to go out for game of Tennis outside. Now the question is how would one decide whether it is ideal to go out for a game of tennis. Now this depends upon various factors like time, weather, temperature etc. We call these factors as features which will influence our decision. If you could record all the factors and decision you took, you could get a table something like this.

Day	Outlook	Temp.	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No

Data collected of last 14 days

With this table, other people would be able to use your intuition to decide whether they should play tennis by looking up what you did given a certain weather pattern, but after just 14 days, it's a little unwieldy to match your current weather situation with one of the rows in the table. We could represent this tabular data in the form of tree.



Creating tree from the data

Here all the information is represented in the form of tree. The rectangular box represents the node of the tree. Splitting of data is done by asking question to the node. The branches represents various possible known outcome obtained by asking the question on the node. The end nodes are the leafs. They represent various classes in which the data can be classified. The two classes in this example are Yes and No. Thus to obtain the class/final output, ask the question to the node and using the answer travel through branch until one reaches the leaf node.

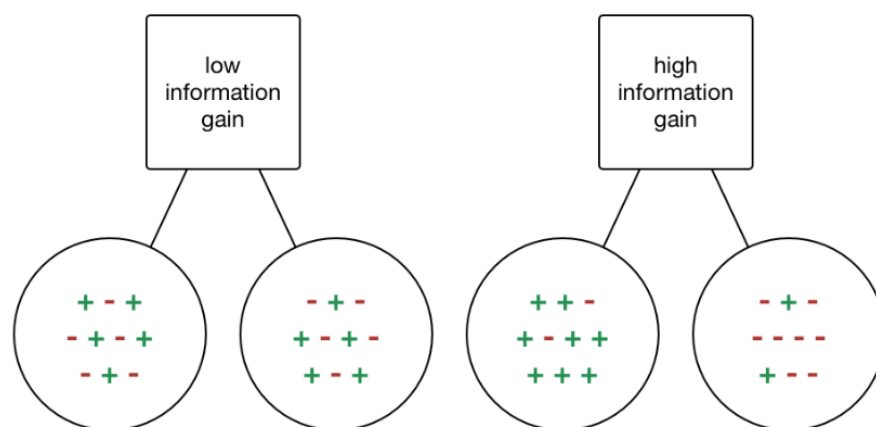
Algorithm

1. Start with a training data set which we'll call S. It should have attributes and classification.
2. Determine the **best attribute** in the dataset. (We will go over the definition of best attribute)
3. Split S into subset that contains the possible values for the best attribute.
4. Make decision tree node that contains the best attribute.
5. Recursively generate new decision trees by using the subset of data created from step 3 until a stage is reached where you cannot classify the data further. Represent the class as leaf node.

Deciding the "BEST ATTRIBUTE"

Now the most important part of Decision Tree algorithm is deciding the best attribute. But what does 'best' actually mean?

In Decision Tree algorithm, the best mean the attribute which has most **information gain**.



The left split has less information gain as the data is split on two classes which has almost equal '+' and '-' examples. While the split on the right has more '+' example in one class and more '-' example in the other class. In order to calculate best attribute we will use **Entropy**.

Entropy

In machine learning sense and especially in this case Entropy is the measure of homogeneity in the data. Its value ranges from 0 to 1. Its value is close to 0 if all the example belongs to same class and is close to 1 if there is almost equal split of the data into different classes. Now the formula to calculate entropy is :

$$Entropy(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

Here p_i represents the proportion of the data with i th classification and c represents the different types of classification.

Now **Information Gain** measure the reduction in entropy by classifying the data on a particular attribute. The formula to calculate **Gain** by splitting the data on Dataset 'S' and on the attribute 'A' is :

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Here $Entropy(S)$ represents the entropy of the dataset and the second term on the right is the weighted entropy of the different possible classes obtain after the split. Now the goal is to maximize this **information gain**. The attribute which has the maximum information gain is selected as the parent node and successively data is split on the node.

Entropy of the dataset ($Entropy(S)$) is :

$$Entropy([9+, 5-]) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = .940$$

Now to calculate **Information Gain**

$$\text{Values}(\text{Outlook}) = \text{Sunny}, \text{Overcast}, \text{Rain}$$

$$S = [9+, 5-]$$

$$S_{\text{Sunny}} = [2+, 3-]$$

$$S_{\text{Overcast}} = [4+, 0-]$$

$$S_{\text{Rain}} = [3+, 2-]$$

$$\begin{aligned} \text{Gain}(S, \text{Outlook}) &= \text{Entropy}(S) \\ &\quad - (5/14)\text{Entropy}(S_{\text{Sunny}}) \\ &\quad - (4/14)\text{Entropy}(S_{\text{Overcast}}) \\ &\quad - (5/14)\text{Entropy}(S_{\text{Rain}}) \\ &= .940 - (5/14).971 - (4/14)0 - (5/14).971 \\ &= .247 \end{aligned}$$

$$\begin{aligned} \text{Gain}(S, \text{Temperature}) &= \text{Entropy}(S) \\ &\quad - (4/14)\text{Entropy}(S_{\text{Hot}}) \\ &\quad - (6/14)\text{Entropy}(S_{\text{Mild}}) \\ &\quad - (4/14)\text{Entropy}(S_{\text{Cool}}) \\ &= .940 - (4/14)1 - (6/14).811 - (4/14).971 \\ &= .029 \end{aligned}$$

$$\begin{aligned} \text{Gain}(S, \text{Humidity}) &= \text{Entropy}(S) \\ &\quad - (7/14)\text{Entropy}(S_{\text{High}}) \\ &\quad - (7/14)\text{Entropy}(S_{\text{Normal}}) \\ &= .940 - (7/14).985 - (7/14).592 \\ &= .152 \end{aligned}$$

$$\begin{aligned} \text{Gain}(S, \text{Wind}) &= \text{Entropy}(S) \\ &\quad - (8/14)\text{Entropy}(S_{\text{Weak}}) \\ &\quad - (6/14)\text{Entropy}(S_{\text{Strong}}) \\ &= .940 - (8/14).811 - (6/14)1 \\ &= .048 \end{aligned}$$

Applying the formula of information gain on all different attributes.

As we can see Outlook attribute has the maximum information gain and hence it is placed at top of the tree.

Information gain is a decrease in entropy. It computes the difference between entropy before split and average entropy after split of the dataset based on given attribute values. ID3 (Iterative Dichotomiser) decision tree algorithm uses information gain.

Mathematically, IG is represented as:

$$\text{Information Gain}(T,X) = \text{Entropy}(T) - \text{Entropy}(T, X)$$

$$\begin{aligned} \text{IG}(\text{PlayGolf}, \text{Outlook}) &= E(\text{PlayGolf}) - E(\text{PlayGolf}, \text{Outlook}) \\ &= 0.940 - 0.693 \\ &= 0.247 \end{aligned}$$

In a much simpler way, we can conclude that:

$$\text{Information Gain} = \text{Entropy}(\text{before}) - \sum_{j=1}^K \text{Entropy}(j, \text{after})$$

Where “before” is the dataset before the split, K is the number of subsets generated by the split, and (j, after) is subset j after the split.

3 — Gini Index

You can understand the Gini index as a cost function used to evaluate splits in the dataset. It is calculated by subtracting the sum of the squared probabilities of each class from one. It favors larger partitions and easy to implement whereas information gain favors smaller partitions with distinct values.

$$\text{Gini} = 1 - \sum_{i=1}^C (p_i)^2$$

Gini Index works with the categorical target variable “Success” or “Failure”. It performs only Binary splits.

Higher value of Gini index implies higher inequality, higher heterogeneity

Steps to Calculate Gini index for a split

1. Calculate Gini for sub-nodes, using the above formula for success(p) and failure(q) (p^2+q^2).
2. Calculate the Gini index for split using the weighted Gini score of each node of that split.

CART (Classification and Regression Tree) uses the Gini index method to create split points.

4 — Gain ratio

Information gain is biased towards choosing attributes with a large number of values as root nodes. It means it prefers the attribute with a large number of distinct values.

C4.5, an improvement of ID3, uses Gain ratio which is a modification of Information gain that reduces its bias and is usually the best option. Gain ratio overcomes the problem with information gain by taking into account the number of branches that would result before making the split. It corrects information gain by taking the intrinsic information of a split into account.

Let us consider if we have a dataset that has users and their movie genre preferences based on variables like gender, group of age, rating, blah, blah. With the help of information gain, you split at 'Gender' (assuming it has the highest information gain) and now the variables 'Group of Age' and 'Rating' could be equally important and with the help of gain ratio, it will penalize a variable with more distinct values which will help us decide the split at the next level.

$$\text{Gain Ratio} = \frac{\text{Information Gain}}{\text{SplitInfo}} = \frac{\text{Entropy (before)} - \sum_{j=1}^K \text{Entropy}(j, \text{after})}{\sum_{j=1}^K w_j \log_2 w_j}$$

Where "before" is the dataset before the split, K is the number of subsets generated by the split, and (j, after) is subset j after the split.

5 — Reduction in Variance

Reduction in variance is an algorithm used for continuous target variables (regression problems). This algorithm uses the standard formula of variance to choose the best split. The split with lower variance is selected as the criteria to split the population:

$$\text{Variance} = \frac{\sum (X - \bar{X})^2}{n}$$

Above X-bar is the mean of the values, X is actual and n is the number of values.

Steps to calculate Variance:

1. Calculate variance for each node.
2. Calculate variance for each split as the weighted average of each node variance.

6— Chi-Square

The acronym *CHAID* stands for *Chi-squared Automatic Interaction Detector*. It is one of the oldest tree classification methods. It finds out the statistical significance between the differences between sub-nodes and parent node. We measure it by the sum of squares of standardized differences between observed and expected frequencies of the target variable.

It works with the categorical target variable "Success" or "Failure". It can perform two or more splits. Higher the value of Chi-Square higher the statistical significance of differences between sub-node and Parent node.

It generates a tree called CHAID (Chi-square Automatic Interaction Detector).

Mathematically, Chi-squared is represented as:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Where:

χ^2 = Chi Square obtained

$\sum$ = the sum of

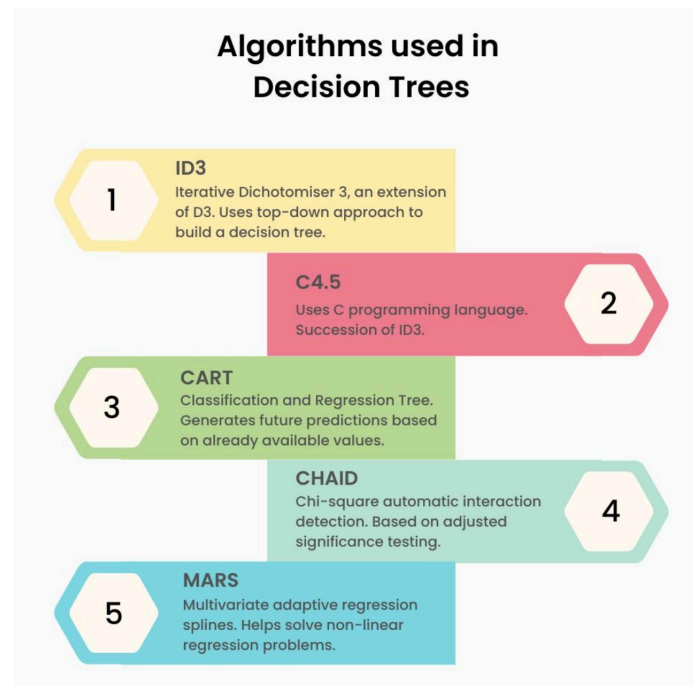
O = observed score

E = expected score

Steps to Calculate Chi-square for a split:

1. Calculate Chi-square for an individual node by calculating the deviation for Success and Failure both
2. Calculated Chi-square of Split using Sum of all Chi-square of success and Failure of each node of the split

3 — Algorithms used in Decision Trees



The following are the algorithms used in decision trees.

ID3

ID3 or Iterative Dichotomiser 3 is an algorithm used to build a decision tree by employing a top-down approach. The tree is built from the top and each iteration with the best feature helps create a node.

Here are the steps:

- The root node is the start point known as a set S .
- Each iteration of the algorithm will iterate through unused attributes of the root node and calculate the information gain (IG) and entropy (S).
- It will select the attribute with the tiniest entropy or higher information gain.
- We divide set S by choosing the attribute to produce the data subset.
- The algorithm will continue if there is no repetition in the attributes chosen.

C4.5

The C4.5 algorithm is an improved version of ID3. C in the algorithm indicates that it uses C programming language and 4.5 is the algorithm's version. It is one of the more popular algorithms for machine learning. It is also used as a decision tree classifier and to generate a decision tree.

CART

Classification and Regression Tree or CART is a predictive algorithm used to generate future predictions based on already available values. These algorithms serve as a base of machine learning algorithms like bagged decision trees, boosted decision trees, or random forests.

There are marked differences between regression trees and classification trees.

- Regression trees: Predict continuous values depending on information sources or previous data. For instance, to predict the price of an item, previous data needs to be analyzed.
- Classification trees: Determine whether an event occurred. It usually has outcomes as either yes or no. This type of decision tree algorithm is often used in real-world decision-making.

CHAID

Chi-square automatic interaction detection (CHAID) is a tree classification method that finds the importance between the parent nodes and root nodes. It is measured by adding the squares of standardized differences between the expected and observed frequencies of the target variable.

It works using the categorical target variables, Success or Failure, and can work on two or more splits. If the Chi-square value is high, the statistical importance of the variation of the parent node and root nodes will also be high. It will generate CHAID.

MARS

Multivariate adaptive regression splines or MARS is a complex algorithm that helps solve non-linear regression problems. It lets us find a set of linear functions that provide the best prediction. It is a combination of simple linear regression functions.

4 — CART (Classification And Regression Tree) in Machine Learning

1 — CART (Classification And Regression Tree) for Decision Tree

CART is a predictive algorithm used in Machine learning and it explains how the target variable's values can be predicted based on other matters. It is a decision tree where each fork is split into a predictor variable and each node has a prediction for the target variable at the end.

The term CART serves as a generic term for the following categories of decision trees:

- **Classification Trees:** The tree is used to determine which "class" the target variable is most likely to fall into when it is continuous.
- **Regression trees:** These are used to predict a continuous variable's value.

In the decision tree, nodes are split into sub-nodes based on a threshold value of an attribute. The root node is taken as the training set and is split into two by considering the best attribute and threshold value. Further, the subsets are also split using the same logic. This continues till the last pure sub-set is found in the tree or the maximum number of leaves possible in that growing tree.

CART Algorithm

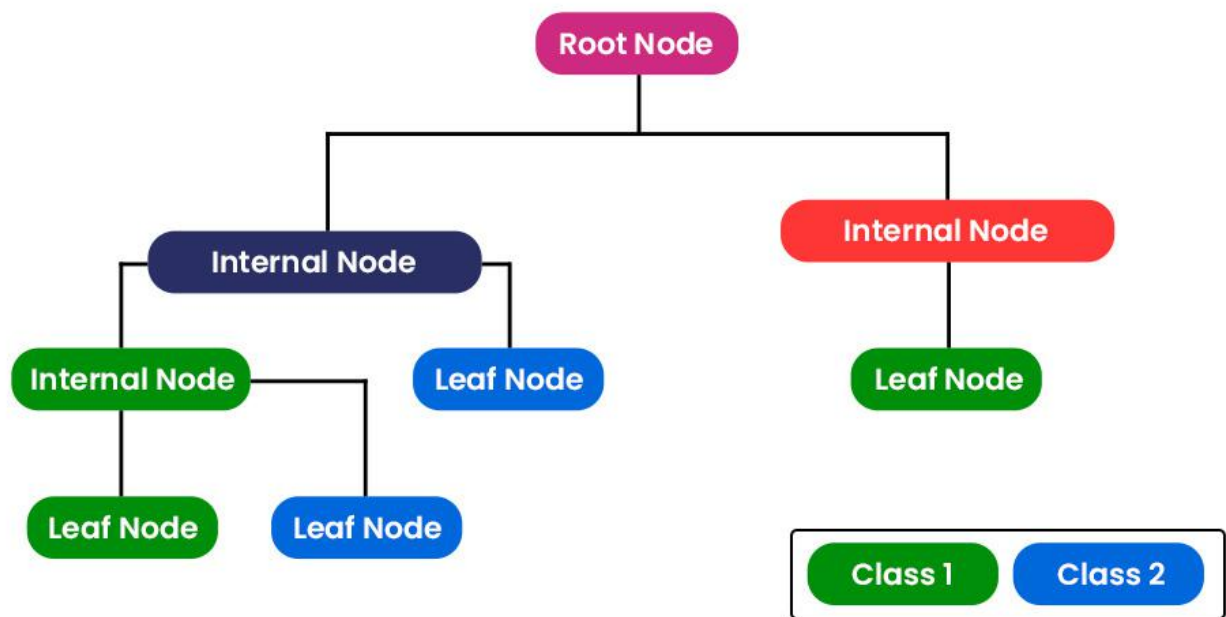
Classification and Regression Trees (CART) is a decision tree algorithm that is used for both classification and regression tasks. It is a supervised learning algorithm that learns from labelled data to predict unseen data.

- **Tree structure:** CART builds a tree-like structure consisting of nodes and branches. The nodes represent different decision points, and the branches represent the possible outcomes of those decisions. The leaf nodes in the tree contain a predicted class label or value for the target variable.
- **Splitting criteria:** CART uses a greedy approach to split the data at each node. It evaluates all possible splits and selects the one that best reduces the impurity of the resulting subsets. For classification tasks, CART uses Gini impurity as the splitting criterion. The lower the Gini impurity, the more pure the subset is. For regression tasks, CART uses residual reduction as the splitting criterion. The lower the residual reduction, the better the fit of the model to the data.
- **Pruning:** To prevent overfitting of the data, pruning is a technique used to remove the nodes that contribute little to the model accuracy. Cost complexity pruning and information gain pruning are two popular pruning techniques. Cost complexity pruning involves calculating the cost of each node and removing nodes that have a negative cost. Information gain pruning involves calculating the information gain of each node and removing nodes that have a low information gain.

2 — How does CART algorithm works?

The CART algorithm works via the following process:

- The best-split point of each input is obtained.
- Based on the best-split points of each input in Step 1, the new "best" split point is identified.
- Split the chosen input according to the "best" split point.
- Continue splitting until a stopping rule is satisfied or no further desirable splitting is available.



CART algorithm uses Gini Impurity to split the dataset into a decision tree. It does that by searching for the best homogeneity for the sub nodes, with the help of the Gini index criterion.

3 — Gini index/Gini impurity

The Gini index is a metric for the classification tasks in CART. It stores the sum of squared probabilities of each class. It computes the degree of probability of a specific variable that is wrongly being classified when chosen randomly and a variation of the Gini coefficient. It works on categorical variables, provides outcomes either "successful" or "failure" and hence conducts binary splitting only.

The degree of the **Gini index** varies from 0 to 1,

- Where 0 depicts that all the elements are allied to a certain class, or only one class exists there.
- Gini index close to 1 means a high level of impurity, where each class contains a very small fraction of elements, and
- A value of $1 - 1/n$ occurs when the elements are uniformly distributed into n classes and each class has an equal probability of $1/n$. For example, with two classes, the Gini impurity is $1 - 1/2 = 0.5$.

Mathematically, we can write Gini Impurity as follows: **Gini** = $1 - \sum_{i=1}^n (p_i)^2$.
where p_i is the probability of an object being classified to a particular class.

In conclusion, **Gini impurity** is the probability of misclassification, assuming independent selection of the element and its class based on the class probabilities.

4 — CART for Classification

A classification tree is an algorithm where the target variable is categorical. The algorithm is then used to identify the "Class" within which the target variable is most likely to fall. Classification trees are used when the dataset needs to be split into classes that belong to the response variable (like yes or no)

For classification in decision tree learning algorithm that creates a tree-like structure to predict class labels. The tree consists of nodes, which represent different decision points, and branches, which represent the possible result of those decisions. Predicted class labels are present at each leaf node of the tree.

5 — How Does CART for Classification Work?

CART for classification works by recursively splitting the training data into smaller and smaller subsets based on certain criteria. The goal is to split the data in a way that minimizes the impurity within each subset. Impurity is a measure of how mixed up the data is in a particular subset. For classification tasks, CART uses Gini impurity

- **Gini Impurity-** Gini impurity measures the probability of misclassifying a random instance from a subset labeled according to the majority class. Lower Gini impurity means more purity of the subset.
- **Splitting Criteria-** The CART algorithm evaluates all potential splits at every node and chooses the one that best decreases the Gini impurity of the resultant subsets. This process continues until a stopping criterion is reached, like a maximum tree depth or a minimum number of instances in a leaf node.

6 — CART for Regression

A Regression tree is an algorithm where the target variable is continuous and the tree is used to predict its value. Regression trees are used when the response variable is continuous. For example, if the response variable is the temperature of the day.

CART for regression is a decision tree learning method that creates a tree-like structure to predict continuous target variables. The tree consists of nodes that represent different decision points and branches that represent the possible outcomes of those decisions. Predicted values for the target variable are stored in each leaf node of the tree.

7 — How Does CART works for Regression?

Regression CART works by splitting the training data recursively into smaller subsets based on specific criteria. The objective is to split the data in a way that minimizes the residual reduction in each subset.

- **Residual Reduction-** Residual reduction is a measure of how much the average squared difference between the predicted values and the actual values for the target variable is reduced by splitting the subset. The lower the residual reduction, the better the model fits the data.
- **Splitting Criteria-** CART evaluates every possible split at each node and selects the one that results in the greatest reduction of residual error in the resulting subsets. This process is repeated until a stopping criterion is met, such as reaching the maximum tree depth or having too few instances in a leaf node.

8 — Pseudo-code of the CART algorithm

```

d = 0, endtree = 0
Node(0) = 1, Node(1) = 0, Node(2) = 0
while endtree < 1
    if Node(2d-1) + Node(2d) + .... + Node(2d+1-2) = 2 - 2d+1
        endtree = 1
    else
        do i = 2d-1, 2d, .... , 2d+1-2
            if Node(i) > -1
                Split tree
            else
                Node(2i+1) = -1
                Node(2i+2) = -1
            end if
        end do
    end if
    d = d + 1
end while

```

9 — CART model representation

CART models are formed by picking input variables and evaluating split points on those variables until an appropriate tree is produced.

Steps to create a Decision Tree using the CART algorithm:

- **Greedy algorithm:** In this the input space is divided using the Greedy method which is known as a recursive binary splitting. This is a numerical method within which all of the values are aligned and several other split points are tried and assessed using a cost function.
- **Stopping Criterion:** As it works its way down the tree with the training data, the recursive binary splitting method described above must know when to stop splitting. The most frequent halting method is to utilize a minimum amount of training data allocated to every leaf node. If the count is smaller than the specified threshold, the split is rejected and also the node is considered the last leaf node.
- **Tree pruning:** Decision tree's complexity is defined as the number of splits in the tree. Trees with fewer branches are recommended as they are simple to grasp and less prone to cluster the data. Working through each leaf node in the tree and evaluating the effect of deleting it using a hold-out test set is the quickest and simplest pruning approach.
- **Data preparation for the CART:** No special data preparation is required for the CART algorithm.

10 — Decision Tree CART Implementations

Here is the code implements the CART algorithm for classifying fruits based on their color and size. It first encodes the categorical data using a LabelEncoder and then trains a CART classifier on the encoded data. Finally, it predicts the fruit type for a new instance and decodes the result back to its original categorical value.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Define the features and target variable
features = [
    ["red", "large"],
    ["green", "small"],
    ["red", "small"],
    ["yellow", "large"],
    ["green", "large"],
    ["orange", "large"],
]
target_variable = ["apple", "lime", "strawberry", "banana", "grape", "orange"]

# Flatten the features list for encoding
flattened_features = [item for sublist in features for item in sublist]

# Use a single LabelEncoder for all features and target variable
le = LabelEncoder()
le.fit(flattened_features + target_variable)

# Encode features and target variable
encoded_features = [le.transform(item) for item in features]
encoded_target = le.transform(target_variable)

# Create a CART classifier
clf = DecisionTreeClassifier()

# Train the classifier on the training set
clf.fit(encoded_features, encoded_target)

# Predict the fruit type for a new instance
new_instance = ["red", "large"]
encoded_new_instance = le.transform(new_instance)
predicted_fruit_type = clf.predict([encoded_new_instance])
decoded_predicted_fruit_type = le.inverse_transform(predicted_fruit_type)
print("Predicted fruit type:", decoded_predicted_fruit_type[0])
```

Output:

Predicted fruit type: apple

11 — POPULAR CART-BASED ALGORITHMS

- **CART (Classification and Regression Trees):** The original algorithm that uses binary splits to build decision trees.
- **C4.5 and C5.0:** Extensions of CART that allow for multiway splits and handle categorical variables more effectively.
- **Random Forests:** Ensemble methods that use multiple decision trees (often CART) to improve predictive performance and reduce overfitting.
- **Gradient Boosting Machines (GBM):** Boosting algorithms that also use decision trees (often CART) as base learners, sequentially improving model performance.

12 — Advantages of CART

- Results are simplistic.
- Classification and regression trees are Nonparametric and Nonlinear.
- Classification and regression trees implicitly perform feature selection.
- Outliers have no meaningful effect on CART.
- It requires minimal supervision and produces easy-to-understand models.

13 — Limitations of CART

- Overfitting.
- High Variance.
- low bias.
- the tree structure may be unstable.

14 — Applications of the CART algorithm

- For quick Data insights.
- In Blood Donors Classification.
- For environmental and ecological data.
- In the financial sectors.

5 — Advantages and Disadvantages of Decision Trees

1 — Advantages

1. Compared to other algorithms decision trees requires less effort for data preparation during pre-processing.
2. A decision tree does not require normalization of data.
3. A decision tree does not require scaling of data as well.
4. Missing values in the data also do NOT affect the process of building a decision tree to any considerable extent.
5. A Decision tree model is very intuitive and easy to explain to technical teams as well as stakeholders.

2 — Disadvantage

1. A small change in the data can cause a large change in the structure of the decision tree causing instability.
 2. For a Decision tree sometimes, calculation can go far more complex compared to other algorithms.
 3. Decision tree often involves higher time to train the model.
 4. Decision tree training is relatively expensive as the complexity and time has taken are more.
 5. The Decision Tree algorithm is inadequate for applying regression and predicting continuous values.
-